

Design patterns & principii

1. HIGH COHESION (Coeziune ridicata)

- O clasa trebuie sa faca un singur lucru si sa il faca bine. Toate metodele si attributele ei trebuie sa lucreze impreuna pentru un scop comun.

2. LOW COUPLING (Cuplare SCAZUTA)

Clasele trebuie sa fie cat mai independente una de alta. Daca modifici ceva intr-o clasa, nu ar trebui sa "crape" programul in alte locuri.

- Se face folosind **INCAPSULAREA (PRIVATE)** si **INTERFETE**.

PRINCIPIILE SOLID

S - Single Responsibility Principle (SRP)

- **Definiție:** O clasă trebuie să aibă o singură responsabilitate (un singur motiv de a fi modificată).
- *Exemplu:* Nu pune metoda `printeazaFactura()` în clasa `Factura`. Clasa `Factura` calculează totalul, o altă clasă `Imprimanta` se ocupă de printare. Altfel, dacă schimbi formatul hârtiei, modifici clasa `Factura` (ceea ce e greșit).

O - Open/Closed Principle (OCP)

- **Definiție:** Entitățile software trebuie să fie **deschise pentru extindere**, dar **închise pentru modificare**.
- *Traducere:* Dacă vrei funcționalitate nouă, nu modifica codul vechi (care merge deja). Creează o clasă nouă care o extinde pe cea veche.
- *Exemplu:* Ai `CalculSalar`. Vrei să adaugi un bonus. Nu intri cu `if` în metoda veche. Faci class `CalculSalarCuBonus` extends `CalculSalar`.

L - Liskov Substitution Principle (LSP)

- **Definiție:** Obiectele clasei derivate (Copil) trebuie să poată înlocui obiectele clasei de bază (Părinte) fără să strice programul.
- *Exemplu clasic (Așa NU):* Pătratul moștenește Dreptunghiul. Dacă setezi latura la Pătrat, se schimbă și lungimea și lățimea. Dacă codul tău se aștepta la un Dreptunghi unde lungimea și lățimea sunt independente, Pătratul va strica logica.

I - Interface Segregation Principle (ISP)

- **Definiție:** Mai bine mai multe interfețe mici și specifice decât una mare și generală.
- *Logica:* Un client nu trebuie forțat să depindă de metode pe care nu le folosește.
- *Exemplu:* Ai interfața Muncitor cu metodele muncesta() și mananca(). Robotul implementează Muncitor. Dar robotul nu mănâncă! Trebuie spartă interfața în Muncitor și FiintaVie.

D - Dependency Inversion Principle (DIP)

- **Definiție:** Modulele de nivel înalt nu trebuie să depindă de modulele de nivel jos. Ambele trebuie să depindă de abstractizări (interfețe).
- *Exemplu:* Clasa Magazin nu trebuie să aibă în ea new BazaDeDateMySQL(). Ea trebuie să aibă o referință către o interfață BazaDeDate. Astfel, poți schimba MySQL cu Oracle fără să modifice Magazin.